背景知识

认识一个人

- 雷霄骅 中国传媒大学的博士生 通信与信息系统专业
- 博客 https://blog.csdn.net/leixiaohua1020?type=blog

- 对于音视频，如果不认识这个人，就相当于你没有入门

生活中的音视频



这些电影文件不同的格式，

用不同的后缀表示

这些格式代表封装格式

封装格式 即把视频数据和音频数据打包成一个文件的规范

不同的封装格式之间差距不大，

各有优劣

如何查看视频采用的视音频技术



什么是视频？什么是帧率？

- 先只考虑画面
- 通俗的讲， 就是连续时间内播放连续画面，只要切换画面频率足够高，由于人眼惰性，会认为画面是移动的
- 比如常见的视频， 是1秒钟25或30张画面，这个数字我们叫帧率，就是通常所说的fps(frames per second)
- 帧率的多少会影响, 画面观看的连续度, 同时会使文件大小变大

什么是分辨率?

- 横向像素数和纵向像素数的乘积
- 常见的 超高清 1080p 也就是 1920*1080,

也就是1张画面长是1920像素, 宽是1080像素 , 长宽可以形成一个平面空间, 空间上有1920*1080个点, 就是我们所谓的像素点

比如一个像素点是 16位数据表示的(BMP或PNG格式图片), 那么一个1080p的图片大小是可以算出来的(大概是4MB).

视频播放器原理

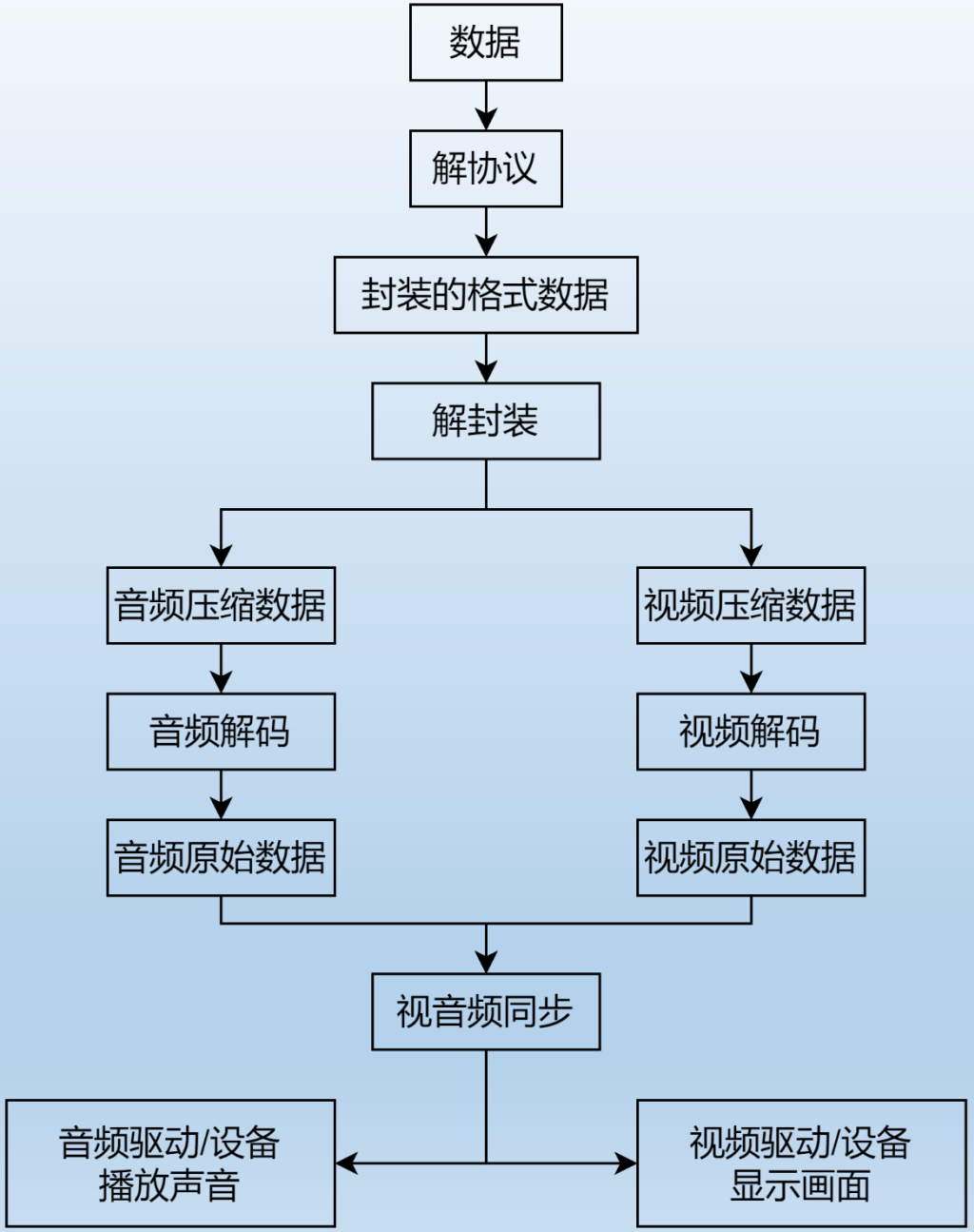
视音频技术主要包含以下几点:

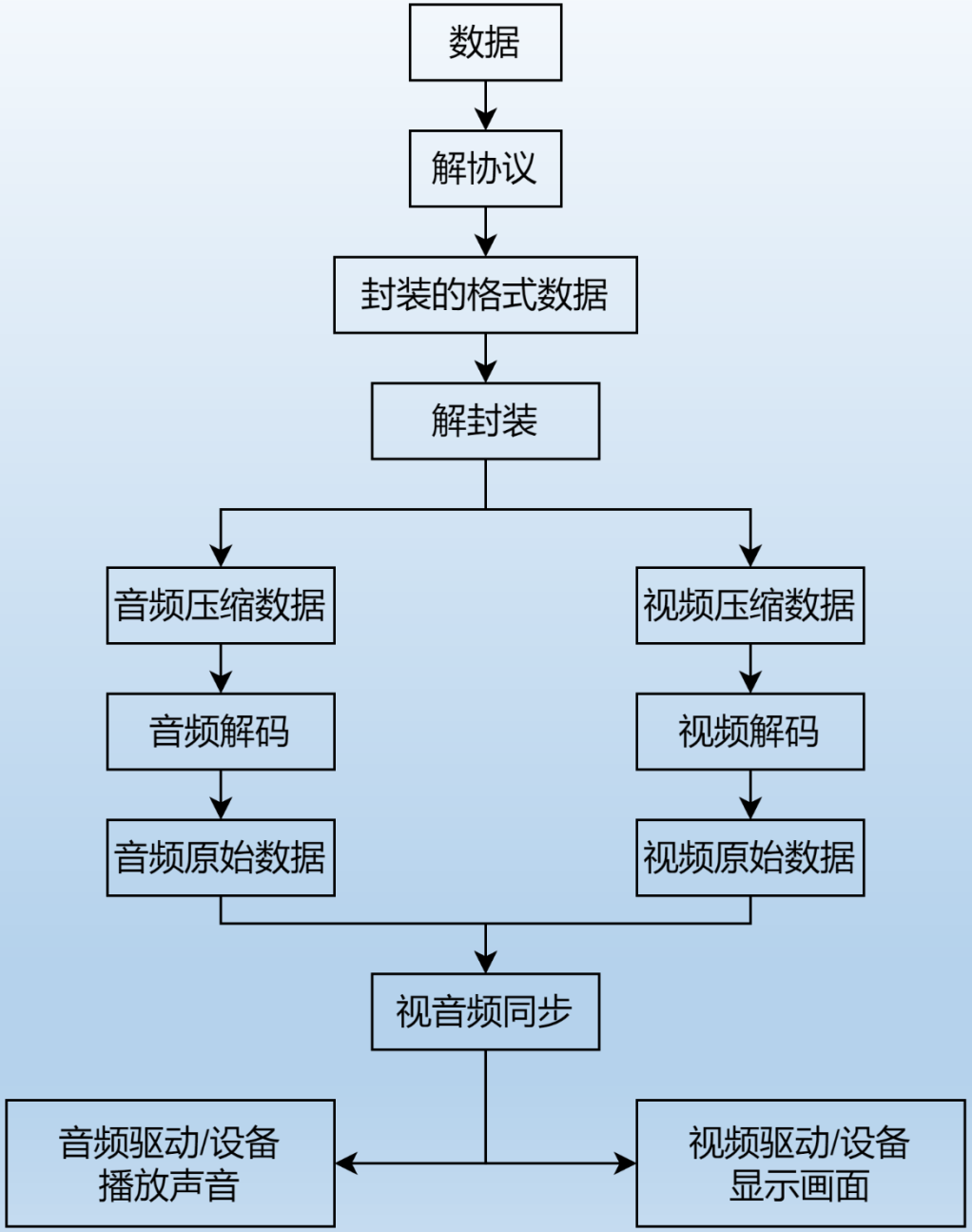
- 封装技术
- 视频压缩编码技术以及音频压缩编码技术。
- 如果考虑到网络传输的话，还包括流媒体协议技术。

播放流程

视频播放器播放一个视频文件

- 解协议(来自网络需要)
- 解封装
- 解码视音频
- 视音频同步





解协议

将流媒体协议的数据，解析为标准的相应的封装格式数据

解封装

将输入的封装格式的数据，分离成音频视频流压缩编码数据

解码

将视/音频压缩编码数据，解码成为非压缩的视频/音频原始数据

视音频同步

根据解封装模块处理过程中获取到的参数信息，
同步解码出来的视频和音频数据，
并将视频音频数据送至系统的显卡和声卡播放出来。

封装格式根据协议，例如MP3，AVI等，FLV，AVI等等
封装格式根据协议，例如MP3，AVI等，FLV，AVI等等
这些协议都将视音频数据封装成一定的格式，
这些协议都将视音频数据封装成一定的格式，
例如FLV将视频数据和音频数据按一定的格式放到一起。
例如FLV将视频数据和音频数据按一定的格式放到一起。
通过传输协议传输数据。经过解协议操作后，
通过传输协议传输数据。经过解协议操作后，
输出FLV格式的数据。
输出FLV格式的数据。

封装格式根据协议，例如MP3，AVI等，FLV，AVI等等
这些协议都将视音频数据封装成一定的格式，
例如FLV将视频数据和音频数据按一定的格式放到一起。
通过传输协议传输数据。经过解协议操作后，

流媒体协议

名称	推出机构	传输层协议	客户端	目前领域
RTSP+RTP	IETF	TCP+UDP	VLC WMP	IPTV 以及各大直播
RTMP	Adobe	TCP	Flash	互联网直播
HTTP	WWW+IETF	TCP	Flash	互联网点播

封装格式

名称	推出机构	流媒体	支持的视频编码	支持的音频编码	目前领域
AVI	微软	不支持	几乎所有格式	几乎所有格式	BT下载影视
MP4	MPEG	支持	MPEG-2, MPEG-4, H.264, H.263等	AAC, MPEG-1 Layers I, II, III, AC-3等	互联网视频网站
TS	MPEG	支持	MPEG-1, MPEG-2, MPEG-4, H.264	MPEG-1 Layers I, II, III, AAC,	IPTV，数字电视
FLV	Adobe	支持	Sorenson, VP6, H.264	MP3, ADPCM, Linear PCM, AAC等	互联网视频网站
MKV	CoreCodec	支持	几乎所有格式	几乎所有格式	互联网视频网站
RMVB	Real NetWorks	支持	RealVideo 8, 9, 10	AAC, Cook Codec, RealAudio Lossless	BT下载影视

视频编码

- 将视频像素数据（RGB，YUV等）压缩成为视频码流，从而降低视频的数据量。

名称	推出机构
HEVC(H.265)	MPEG
H.264	MPEG
MPEG4	MPEG
VP9	Google

主流编码标准
H.264 , H.265

音频编码

- 将音频采样数据（PCM等）压缩成为音频码流，从而降低音频的数据量。

名称	推出机构	领域
AAC	MPEG	各个领域
AC-3	Dolby	电影
MP3	MPEG	各个领域(旧)
WMA	Microsoft	微软平台

编码（encode）：

通过特定的压缩技术，将某个视频的视频流格式转换成另一种视频格式的视频流方式。

解码（decode）：

通过特定的解压缩技术,将某个视频格式的视频流转换成另一种视频格式的视频流方式。

转码（transcode）：

视频转码技术将视频信号从一种格式转换成另一种格式。

音视频基本概念　编码

视频：　YUV420/422->H264
RGB888->H264
YUV420->h263

音频：　PCM(原始)->AAC
PCM(原始)->G726
PCM（原始）->G711

音视频基本概念　转码

视频： 改变分辨率（resolution）
改变帧率（frame rate）
改变比特率 等编码参数

音频： 改变采用率(sample rate)
改变通道数(channels)
改变位宽(sample format)

封装（mux）：复用，按一定格式组织原视音视频流

解封装（demux）：解复用，按一定格式解析出原始音视频流

rtsp流： RTSP（Real Time Streaming Protocol），RFC2326，实时流传输协议，是TCP/IP协议体系中的一个 应用层协议。

rtmp流： Real Time Messaging Protocol（实时消息传输协议），是adobe公司的协议

hls流：HLS 是Apple的动态码率自适应技术。主要用于PC和Apple终端的音视 频服务。包括一个m3u8索引文件，TS媒体分片文件

推模式： 当通知消息来之时，把所有相关信息都通过参数的形式“推给”观察者。

拉模式： 当通知消息来之时，通知的函数不带任何相关的信息，而是要观察者主动去“拉”信息

实时流： Real Time stream 实时传输的音视频流

实战开发

针对视频播放部分, 音频和视频的解码工具是FFMPEG, 对文件或网络资源进行解码
视频的播放采用YUV数据转换为RGB, 绘图显示到控件

为什么采用上面的解码工具?
首先, 我们没有要完全手写一个解码h264的项目, 而是借用第三方开发库, 解码h264.
主流的h264编码器有 x264 t264等
对于新手来说，由于对h.264不是很了解，因此在使用x264上有点困难，出了问题通常不知道如何解决。并且这些库都只能编码一种格式，所以一开始不推荐使用。
而是推荐使用FFMPEG。

FFMPEG是一个集成了各种编解码器的库，可以说是一个全能型的工具，从视频采集、视频编码到视频传输（包括RTP、RTCP、RTMP、RTSP等等协议）都可以直接使用FFMPEG来完成，更重要的一点FFMPEG是跨平台的，Windows、Linux、Android、IOS这些主流系统通吃。因此初期强烈建议直接使用FFMPEG。

引入ffmpeg 并试验

Qt Creator中修改工程配置文件 testFFmpeg.pro：在里面加入：

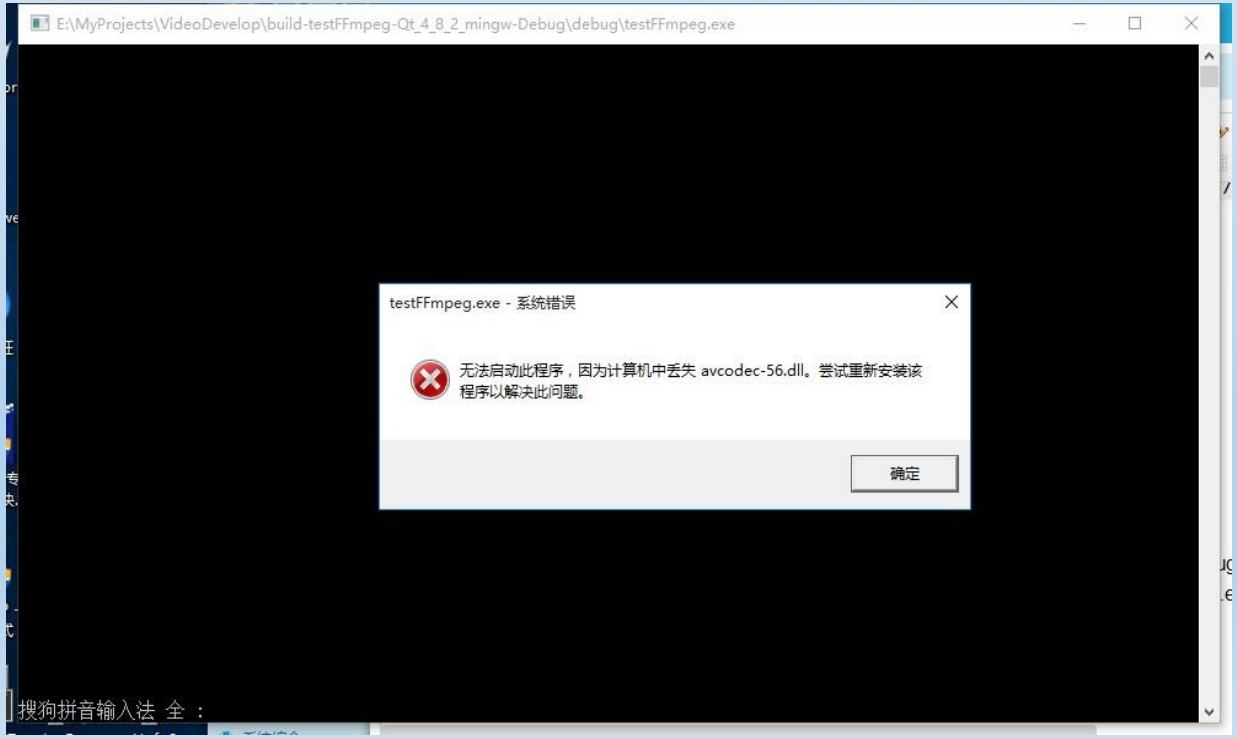
```
INCLUDEPATH += $$PWD/ffmpeg-4.2.2/include

LIBS += $$PWD/ffmpeg-4.2.2/lib/avcodec.lib\
        $$PWD/ffmpeg-4.2.2/lib/avdevice.lib\
        $$PWD/ffmpeg-4.2.2/lib/avfilter.lib\
        $$PWD/ffmpeg-4.2.2/lib/avformat.lib\
        $$PWD/ffmpeg-4.2.2/lib/avutil.lib\
        $$PWD/ffmpeg-4.2.2/lib/postproc.lib\
    $$PWD/ffmpeg-4.2.2/lib/swresample.lib\
    $$PWD/ffmpeg-4.2.2/lib/swscale.lib
./  ---  $$PWD
```

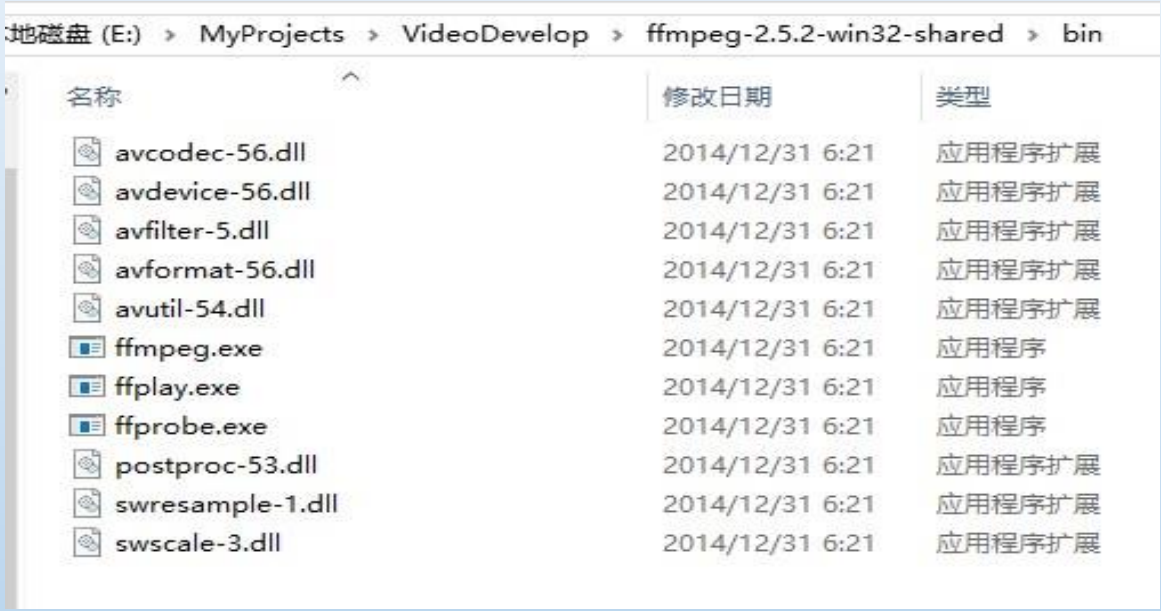
引入头文件

```
extern "C"
{
#include "libavcodec/avcodec.h"
#include "libavformat/avformat.h"
#include "libswscale/swscale.h"
#include "libavdevice/avdevice.h"
}
```

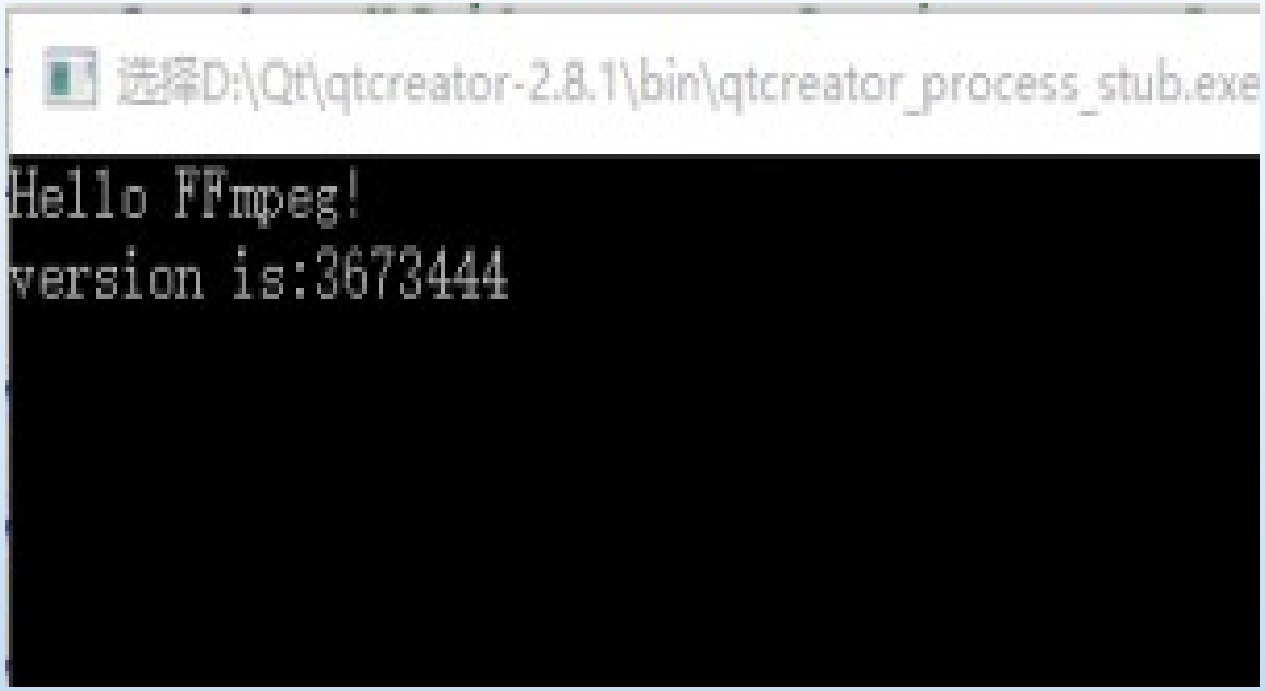
```
int main(int argc, char *argv[])
{
    //这里简单的输出一个版本号
    cout << "Hello FFmpeg!" << endl;
    av_register_all();
    unsigned version = avcodec_version();
    cout << "version is:" << version;
    return 0;
}
```



原来忘记把ffmpeg的dll文件放到里面了。
打开bin目录，里面都是dll文件，这些就是运行时需要的动态库，



将这些dll复制到，刚才我们生成exe的目录下（要和exe放在一起）
现在重新打开Qt Creator，运行，这次终于对了



至此，就成功了引用了FFMPEG了。

总结：
引入ffmpeg的过程

1.添加引入库
2.添加依赖路径
3.添加头文件
4.exe目录下添加dll文件

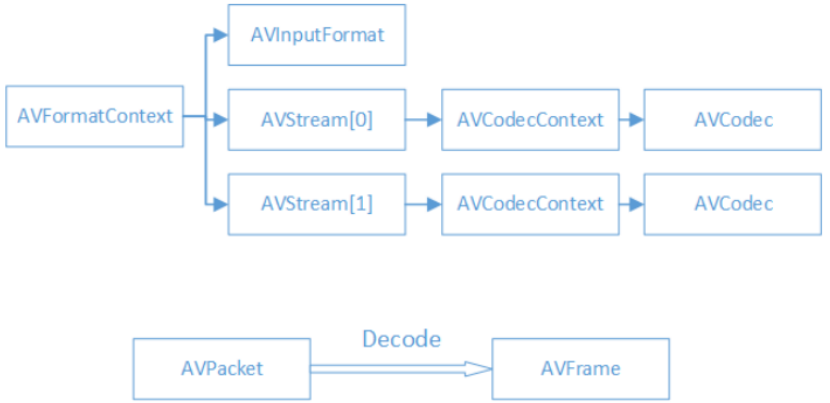
FFMPEG解码视频文件

- FFmpeg解码函数简介
  - av_register_all()：注册所有组件。
  - avformat_open_input()：打开输入视频文件。
  - avformat_find_stream_info()：获取视频文件信息。
  - avcodec_find_decoder()：查找解码器。
  - avcodec_open2()：打开解码器。
  - av_read_frame()：从输入文件读取一帧压缩数据。
  - avcodec_decode_video2()：解码一帧压缩数据。
  - avcodec_close()：关闭解码器。
  - avformat_close_input()：关闭输入视频文件。

- FFmpeg数据结构简介
  - AVFormatContext
    - 封装格式上下文结构体，也是统领全局的结构体，保存了视频文件封装格式相关信息。
  - AVInputFormat
    - 每种封装格式（例如FLV，MKV，MP4，AVI）对应一个该结构体。
  - AVStream
    - 视频文件中每个视频（音频）流对应一个该结构体。
  - AVCodecContext
    - 编码器上下文结构体，保存了视频（音频）编解码相关信息。
  - AVCodec
    - 每种视频（音频）编解码器(例如H.264解码器)对应一个该结构体。
  - AVPacket
    - 存储一帧压缩编码数据。
  - AVFrame
    - 存储一帧解码后像素（采样）数据。

FFmpeg解码的数据结构

- FFmpeg解码的数据结构如下所示



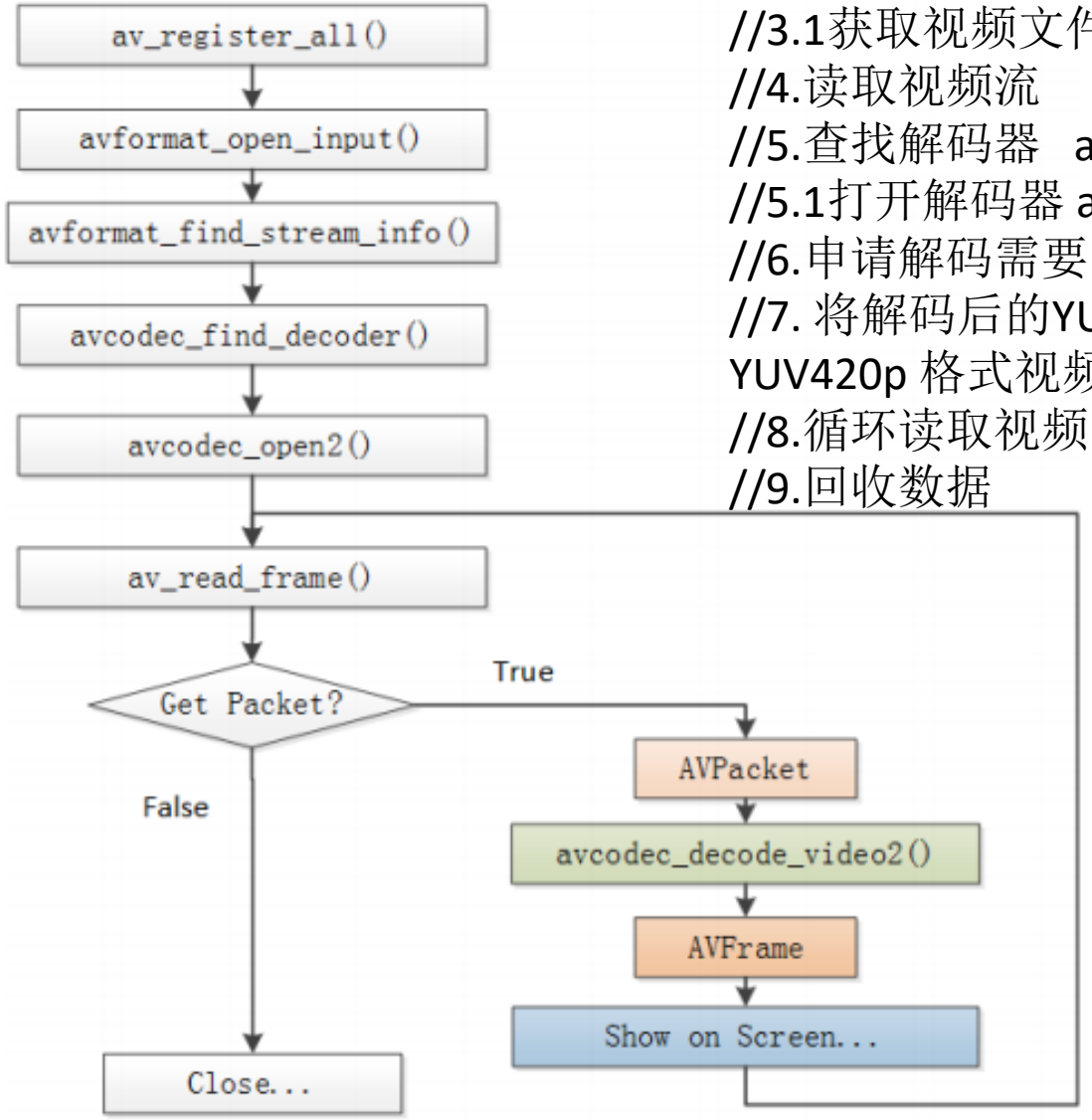
- FFmpeg数据结构分析
  - AVFormatContext
    - iformat：输入视频的AVInputFormat
    - nb_streams ：输入视频的AVStream 个数
    - streams ：输入视频的AVStream []数组
    - duration ：输入视频的时长（以微秒为单位）
    - bit_rate ：输入视频的码率
  - AVInputFormat
    - name：封装格式名称
    - long_name：封装格式的长名称
    - extensions：封装格式的扩展名
    - id：封装格式ID
    - 一些封装格式处理的接口函数

- FFmpeg数据结构分析
  - AVStream
    - id：序号
    - codec：该流对应的AVCodecContext
    - time_base：该流的时基
    - r_frame_rate：该流的帧率
  - AVCodecContext
    - codec：编解码器的AVCodec
    - width，height：图像的宽高（只针对视频）
    - pix_fmt：像素格式（只针对视频）
    - sample_rate：采样率（只针对音频）
    - channels：声道数（只针对音频）
    - sample_fmt：采样格式（只针对音频）
  - AVCodec
    - name：编解码器名称
    - long_name：编解码器长名称
    - type：编解码器类型
    - id：编解码器ID
    - 一些编解码的接口函数

- FFmpeg数据结构分析
  - AVPacket
    - pts：显示时间戳
    - dts ：解码时间戳
    - data ：压缩编码数据
    - size ：压缩编码数据大小
    - stream_index ：所属的AVStream
  - AVFrame
    - data：解码后的图像像素数据（音频采样数据）。
    - linesize：对视频来说是图像中一行像素的大小；对音频来说是整个音频帧的大小。
    - width，height：图像的宽高（只针对视频）。
    - key_frame：是否为关键帧（只针对视频） 。
    - pict_type：帧类型（只针对视频） 。例如I，P，B。

FFmpeg解码的函数

- FFmpeg解码的流程图如下所示

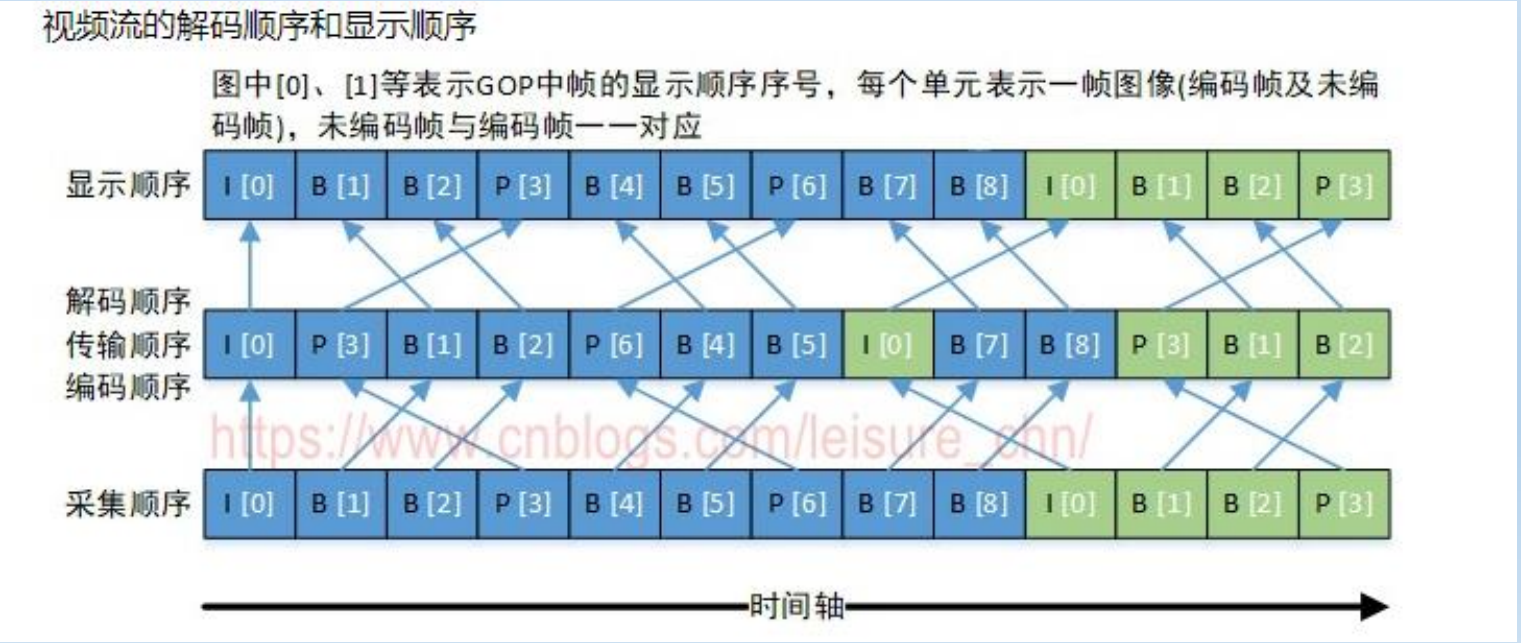


//1.初始化FFMPEG　av_register_all
//2.需要分配一个AVFormatContext，
FFMPEG所有的操作都要通过这个AVFormatContext来进行
可以理解为视频文件指针
//3. 打开视频文件　avformat_open_input()
//3.1获取视频文件信息 avformat_find_stream_info()
//4.读取视频流
//5.查找解码器　avcodec_find_decoder
//5.1打开解码器 avcodec_open2
//6.申请解码需要的结构体 AVFrame 视频缓存的结构体
//7. 将解码后的YUV数据转换成RGB32
YUV420p 格式视频数据-->RGB32--> 图片显示出来
//8.循环读取视频帧, 转换为RGB格式, 抛出信号去控件显示
//9.回收数据

简易同步

I帧/P帧/B帧

I帧：I帧(Intra-coded picture, 帧内编码帧，常称为关键帧)包含一幅完整的图像信息，属于帧内编码图像，
不含运动矢量，在解码时不需要参考其他帧图像。因此在I帧图像处可以切换频道，而不会导致图像丢失或无法解码。
P帧：P帧(Predictive-coded picture, 预测编码图像帧, 常称为差异帧)是帧间编码帧，
利用之前的I帧或P帧进行预测编码。
B帧：B帧(Bi-directionally predicted picture, 双向预测编码图像帧, 常称为双向差异帧)是帧间编码帧，
利用之前和(或)之后的I帧或P帧进行双向预测编码。B帧不可以作为参考帧。B帧具有更高的压缩率，
但需要更多的缓冲时间以及更高的CPU占用率，因此B帧适合本地存储以及视频点播，
而不适用对实时性要求较高的直播系统。



PTS：显示时间戳。
DTS：解码时间戳。

同步采用 PTS

简易同步思路

在播放视频的时候, 我们发现视频较快, 会出现过快, 与时间不同步, 解决办法, 每次解析的时候, 都记录时间(显示时间戳),
然后判断当前视频的时间, 比较真实时间和显示时间, (该方法下,视频会超前),
一旦显示时间更大, 那么就等一下(msleep(10)), 从而解决视频与时间不同步的问题.